BANCO DE DADOS

com Microsoft SQL Server

AULA 12

Operações entre Conjuntos

UNION · UNION ALL · INTERSECT · EXCEPT

UNION | UNION ALL | INTERSECT | EXCEPT | JOIN vs UNION | Regras | Performance | Casos reais

🎯 Objetivos da Aula 12

- ∪ Combinar resultados de múltiplas queries com UNION
- 📋 Dominar as 3 regras obrigatórias de todas as operações
- ⚡ Usar UNION ALL para performance quando não há duplicatas
- VS Distinguir JOIN (combina colunas) de UNION (combina linhas)
- ∩ Identificar elementos comuns com INTERSECT
- 📊 Criar relatórios consolidados com múltiplas fontes de dados
- \ Encontrar diferenças com EXCEPT
- 💻 Aplicar todas as operações no SISTEMA_VENDAS com dados reais

🔄 Revisão — Aula 11: JOINs Avançados

❓ O que LEFT JOIN retorna a mais que INNER JOIN?

✅ Todos os registros da tabela ESQUERDA, mesmo sem correspondência — aparecem com NULL

❓ Como encontrar registros 'órfãos' com LEFT JOIN?

✅ LEFT JOIN + WHERE coluna_direita IS NULL — retorna apenas os sem correspondência

❓ Para que serve o SELF JOIN?

✅ Unir uma tabela com ela mesma — hierarquias (funcionário/gerente) e comparações

❓ Quando usar INNER vs LEFT JOIN?

✅ FK garante par? INNER. Pode não ter par (relação opcional)? LEFT. Regra de ouro!

❓ O que FULL OUTER JOIN retorna?

✅ Todos os registros de AMBAS as tabelas, com NULL nos campos sem correspondência

JOIN vs UNION — Duas Abordagens Complementares

JOIN e UNION são abordagens ortogonais — resolvem problemas diferentes. JOIN combina COLUNAS (expande horizontalmente). UNION combina LINHAS (expande verticalmente).

JOIN — Expande COLUNAS →

id	nome
1	Ana
2	Bruno

+

id	total
1	479
2	7199

→

id	nome
1	Ana
2	Bruno

```
SELECT c.nome, p.total

FROM CLIENTES c JOIN PEDIDOS p ON c.id_cliente=p.id_cliente;
```

Mesmo nº de linhas, mais colunas

UNION — Empilha LINHAS ↓

nome	tipo
Ana	Cliente
Bruno	Cliente
João	Funcionário
Maria	Funcionária

UNION

Mesmo nº de colunas, mais linhas

💡 Regra mnemônica: JOIN é horizontal (+ colunas), UNION é vertical (+ linhas). Pergunte-se: 'Quero mais informação por registro ou mais registros?'

⚠️ As 3 Regras Obrigatórias — UNION, INTERSECT e EXCEPT

Todas as operações de conjunto (UNION, INTERSECT, EXCEPT) exigem as MESMAS 3 regras. Violar qualquer uma gera erro de sintaxe.

1 Regra 1: Mesmo número de colunas

💡 O SQL Server não sabe como combinar 3 colunas com 2 colunas.

✅
```
SELECT nome, estado FROM CLIENTES
UNION SELECT nome, cargo  FROM FUNCIONARIOS;
```

❌
```
SELECT nome, email, estado FROM CLIENTES
-- Msg 205: All queries combined using a UNION must
```

2 Regra 2: Tipos de dados compatíveis (por posição)

💡 Coluna 1 com coluna 1, coluna 2 com coluna 2 — tipos devem ser compatíveis.

✅
```
SELECT nome, id_cliente  FROM CLIENTES
UNION SELECT nome, id_funcionario FROM
FUNCIONARIOS;
```

❌
```
SELECT nome, data_cadastro FROM CLIENTES
-- data (DATE) vs salario (DECIMAL) — incompatível!
```

3 Regra 3: ORDER BY somente no final

💡 ORDER BY é aplicado ao resultado FINAL da operação, não a cada SELECT individual.

✅
```
SELECT nome FROM CLIENTES
UNION SELECT nome FROM FUNCIONARIOS ORDER BY
nome;  -- só aqui!
```

❌
```
SELECT nome FROM CLIENTES ORDER BY nome
SELECT nome FROM FUNCIONARIOS;
```

💻 UNION e UNION ALL — Unindo Resultados

```
-- UNION: elimina duplicatas (mais lento — ordena e compara)
-- UNION ALL: mantém tudo (mais rápido — sem comparação)

-- Ex 1: Lista unificada de todas as pessoas (UNION)
SELECT nome, 'Cliente'     AS tipo, cidade FROM CLIENTES
UNION
SELECT nome, 'Funcionário' AS tipo, NULL   FROM FUNCIONARIOS
ORDER  BY tipo, nome;
-- NULL na cidade dos funcionários — coluna deve existir nas duas partes
-- Ex 2: Relatório consolidado de valores (UNION ALL)
SELECT 'Produto'     AS origem, nome_produto AS descricao,
       preco         AS valor,  'Catálogo'   AS contexto
FROM   PRODUTOS
UNION ALL
SELECT 'Funcionário' AS origem, nome          AS descricao,
       salario       AS valor,  'RH'          AS contexto
FROM   FUNCIONARIOS
ORDER  BY valor DESC;

-- Ex 3: Auditoria de totais — verificar consistência
SELECT 'Pedidos registrados'    AS origem, COUNT(*) AS qtd, SUM(total)   AS soma FROM PEDIDOS
UNION ALL
SELECT 'Itens calculados'       AS origem, COUNT(DISTINCT id_pedido),
       SUM(quantidade*preco_unitario) FROM ITENS_PEDIDO;
-- Se as somas divergirem → inconsistência de dados!
```

💻 INTERSECT — O que Há em Comum

```sql
-- INTERSECT: retorna apenas linhas presentes em AMBAS as queries
-- Elimina duplicatas automaticamente
-- Requer as mesmas 3 regras: mesmas colunas, tipos compatíveis
-- Ex 1: Pessoas que são clientes E funcionários (pelo nome)
SELECT nome FROM CLIENTES
INTERSECT
SELECT nome FROM FUNCIONARIOS;
-- Com nossos dados: provavelmente vazio (nomes diferentes)
-- Ex 2: Categorias que têm produtos E que tiveram vendas
SELECT id_categoria FROM PRODUTOS
INTERSECT
SELECT pr.id_categoria
FROM   PRODUTOS pr
INNER  JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;
-- Retorna: categorias que têm ao menos 1 produto vendido
-- Ex 3: Clientes que compraram em Jan E em outro mês
SELECT id_cliente FROM PEDIDOS
WHERE  MONTH(data_pedido) = 1
INTERSECT
SELECT id_cliente FROM PEDIDOS
WHERE  MONTH(data_pedido) <> 1;
-- Clientes recorrentes: compraram em janeiro E em outros meses
-- Alternativa equivalente com JOIN (mais flexível):
SELECT DISTINCT c.id_cliente FROM PEDIDOS c
WHERE  c.id_cliente IN (SELECT id_cliente FROM PEDIDOS WHERE MONTH(data_pedido)=1)
AND    c.id_cliente IN (SELECT id_cliente FROM PEDIDOS WHERE MONTH(data_pedido)<>1);
```

💻 EXCEPT — Encontrando as Diferenças

```sql
-- EXCEPT: retorna linhas de A que NÃO estão em B
-- A EXCEPT B ≠ B EXCEPT A (não comutativo!)
-- Elimina duplicatas automaticamente

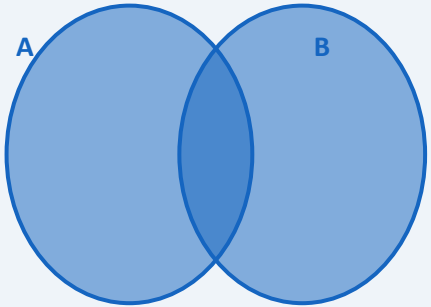
-- Ex 1: Clientes que NUNCA fizeram pedido
SELECT id_cliente, nome FROM CLIENTES
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome
FROM   CLIENTES c
INNER  JOIN PEDIDOS p ON c.id_cliente = p.id_cliente;
-- Com nossos dados: Gabriela e Henrique (sem pedido)

-- Ex 2: Produtos que NUNCA foram vendidos
SELECT id_produto, nome_produto FROM PRODUTOS
EXCEPT
SELECT DISTINCT pr.id_produto, pr.nome_produto
FROM   PRODUTOS    pr
INNER  JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;

-- Ex 3: Categorias SEM nenhum produto vendido
SELECT id_categoria, nome_categoria FROM CATEGORIAS     -- Comparando: EXCEPT vs LEFT JOIN + IS NULL
EXCEPT                                                  -- EXCEPT: mais simples de escrever e ler
SELECT DISTINCT c.id_categoria, c.nome_categoria        -- LEFT JOIN + IS NULL: mais flexível (mais colunas no resultado)
FROM   CATEGORIAS c                                     -- NOT EXISTS: mais performático em grandes volumes
INNER  JOIN PRODUTOS   pr ON c.id_categoria  = pr.id_categoria
INNER  JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;
```

Diagrama Comparativo — UNION, INTERSECT e EXCEPT

∪ UNION

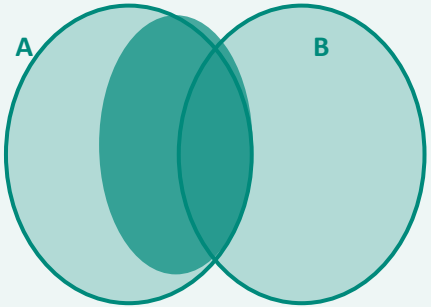



```
UNION / UNION ALL
```

UNION: elimina
UNION ALL: mantém

💼 Clientes + Funcionários
na mesma lista

∩ INTERSECT

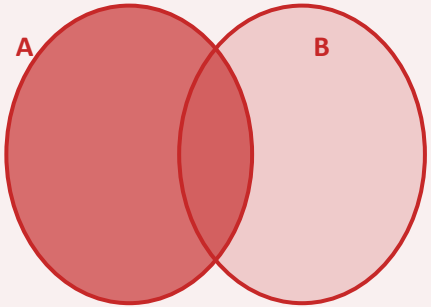



```
INTERSECT
```

Sempre elimina
duplicatas

💼 Categorias com produto
E com venda

∖ EXCEPT




```
EXCEPT
```

Sempre elimina
duplicatas

💼 Clientes sem
nenhum pedido

Tabela Comparativa — Todas as Operações de Conjunto

Característica	UNION	UNION ALL	INTERSECT	EXCEPT
O que retorna?	A ∪ B (sem dup.)	A ∪ B (com dup.)	A ∩ B	A \ B
Elimina duplicatas?	✅ Sim	❌ Não	✅ Sim	✅ Sim
Performance	🐢 Mais lenta	⚡ Mais rápida	Média	Média
Quando usar?	Fontes que podem dup.	Fontes sem duplicatas	Elementos comuns	Diferença/ausência
Mesmo nº colunas?	✅ Obrigatório	✅ Obrigatório	✅ Obrigatório	✅ Obrigatório
Tipos compatíveis?	✅ Obrigatório	✅ Obrigatório	✅ Obrigatório	✅ Obrigatório
ORDER BY onde?	Só no final	Só no final	Só no final	Só no final
Pode combinar N?	✅ Sim	✅ Sim	✅ Sim	✅ Sim

💡 Dica de performance: use UNION ALL sempre que souber que não há duplicatas — é significativamente mais rápido que UNION em grandes volumes.

💻 Casos Avançados — Combinando Operações

```sql
-- CASO 1: Relatório consolidado com UNION ALL + GROUP BY
-- Resumo geral: clientes vs funcionários por estado/localidade
SELECT tipo, COUNT(*) AS qtd, estado FROM (
    SELECT 'Cliente'     AS tipo, estado FROM CLIENTES
    UNION ALL
    SELECT 'Funcionário' AS tipo, NULL   FROM FUNCIONARIOS
) AS pessoas
GROUP  BY tipo, estado
ORDER  BY tipo, estado;
-- CASO 2: EXCEPT + UNION — combinar operações de conjunto
-- Clientes inativos de SP ou RJ (sem pedido) — campanha regional
SELECT id_cliente, nome, 'SP' AS estado FROM CLIENTES WHERE estado='SP'
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome, 'SP'
FROM   CLIENTES c JOIN PEDIDOS p ON c.id_cliente=p.id_cliente
WHERE  c.estado='SP'
UNION
SELECT id_cliente, nome, 'RJ' FROM CLIENTES WHERE estado='RJ'
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome, 'RJ'
FROM   CLIENTES c JOIN PEDIDOS p ON c.id_cliente=p.id_cliente
WHERE  c.estado='RJ';
-- CASO 3: UNION com ORDER e paginação
SELECT nome, preco AS valor, 'Produto' AS tipo FROM PRODUTOS
UNION ALL
SELECT nome, salario, 'Funcionário' FROM FUNCIONARIOS
ORDER  BY valor DESC
OFFSET 0 ROWS FETCH NEXT 5 ROWS ONLY;
```

EXCEPT vs LEFT JOIN vs NOT EXISTS — Três Caminhos

Mesma pergunta: 'Quais clientes nunca fizeram pedido?' — três abordagens diferentes:

EXCEPT

```
SELECT id_cliente, nome
FROM   CLIENTES
EXCEPT
SELECT DISTINCT c.id_cliente, c.nome
FROM   CLIENTES c
JOIN   PEDIDOS p
   ON  c.id_cliente=p.id_cliente;
```

✅ Mais legível e intuitivo

⚠️ Requer mesmas colunas em ambos os lados

💡 Use para análises rápidas e código legível

LEFT JOIN + IS NULL

```
SELECT c.id_cliente, c.nome
FROM   CLIENTES c
LEFT   JOIN PEDIDOS p
    ON c.id_cliente=p.id_cliente
WHERE  p.id_pedido IS NULL;
```

✅ Flexível — pode incluir mais colunas

⚠️ Cuidado com duplicatas em JOIN 1:N

💡 Use quando precisar de colunas extras no resultado

NOT EXISTS

```
SELECT id_cliente, nome
FROM   CLIENTES c
WHERE  NOT EXISTS (
    SELECT 1 FROM PEDIDOS p
    WHERE p.id_cliente=c.id_cliente
);
```

✅ Mais performático em grandes volumes

⚠️ Sintaxe um pouco mais verbosa

💡 Use em produção com tabelas grandes

✏️ Exercícios Propostos — Aula 12

01 — Básico

Use UNION para criar uma lista unificada de todas as pessoas do sistema (clientes + funcionários). Adicione coluna 'tipo' ('Cliente'/'Funcionário'). Ordene por tipo e nome.

02 — Básico

Use UNION ALL para criar um relatório consolidado de valores: produtos (nome, preço, 'Produto') e funcionários (nome, salário, 'Funcionário'). Ordene pelo valor decrescente.

03 — Médio

Use EXCEPT para encontrar: (a) clientes sem pedido, (b) produtos sem venda. Compare com as soluções de LEFT JOIN + IS NULL da Aula 11. Verifique se os resultados são idênticos.

04 — Médio

Use INTERSECT para identificar categorias que têm produtos cadastrados E que tiveram ao menos uma venda. Depois, use EXCEPT para encontrar categorias com produtos mas SEM venda.

05 — Avançado

Crie um relatório de auditoria com UNION ALL: para cada tabela principal (CLIENTES, PRODUTOS, PEDIDOS), mostre o nome da tabela, total de registros (COUNT) e uma métrica relevante (SUM de total, SUM de preco*estoque etc.).

✅ Respostas Comentadas — Exercícios 04 e 05

```sql
-- EX 04a: Categorias COM produtos E COM vendas (INTERSECT)
SELECT id_categoria FROM PRODUTOS
INTERSECT
SELECT pr.id_categoria
FROM   PRODUTOS pr
INNER  JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;
-- EX 04b: Categorias COM produtos MAS SEM venda (EXCEPT)
SELECT id_categoria FROM PRODUTOS
EXCEPT
SELECT pr.id_categoria
FROM   PRODUTOS pr
INNER  JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto;
-- EX 05: Painel de saúde do banco com UNION ALL
SELECT 'CLIENTES'    AS tabela,
       COUNT(*)      AS registros,
       COUNT(email)  AS com_email,
       NULL          AS metrica_valor
FROM   CLIENTES
UNION ALL
SELECT 'PRODUTOS', COUNT(*), COUNT(id_categoria),
       SUM(preco * estoque) FROM PRODUTOS
UNION ALL
SELECT 'PEDIDOS',  COUNT(*), COUNT(CASE WHEN status<>'CANCELADO' THEN 1 END),
       SUM(CASE WHEN status<>'CANCELADO' THEN total ELSE 0 END) FROM PEDIDOS;
```

🏆 Desafio da Aula 12

📋 Contexto: Você foi designado para criar o 'Dashboard de Inteligência de Negócios' do SISTEMA_VENDAS — um conjunto de queries que respondem às perguntas mais críticas da gestão, usando operações de conjunto.

🎯 Queries do dashboard:

D1 Lista VIP: clientes que compraram mais de 1 vez (INTERSECT entre meses) E que gastaram acima da média (subquery). Mostrar nome, total gasto e classificação.

D2 Catálogo inativo: produtos nunca vendidos (EXCEPT) por categoria. Incluir preço, estoque e valor parado em estoque (preco×estoque). Ordenar pelo valor parado.

D3 Relatório de cobertura: usando UNION ALL, mostrar para cada estado: clientes ativos (com pedido), inativos (sem pedido) e percentual de conversão calculado.

D4 Auditoria financeira: UNION ALL comparando totais de PEDIDOS (campo total registrado) vs ITENS_PEDIDO (total calculado). Identificar divergências com EXCEPT.

D5 BÔNUS: Crie uma query com UNION ALL que gere um 'extrato' cronológico: pedidos de clientes E alterações de estoque (simule com UPDATE data), unificados e ordenados por data.

📚 Resumo — Aula 12: Operações entre Conjuntos

∪ UNION

Une resultados eliminando duplicatas. Mais lento (ordena+compara)

⚡ UNION ALL

Une resultados mantendo duplicatas. Mais rápido — prefira quando não há dups

∩ INTERSECT

Retorna só o que existe em AMBAS as queries. Elimina duplicatas

∖ EXCEPT

Retorna o que está em A mas NÃO em B. Não comutativo! A≠B

📋 3 Regras

Mesmo nº colunas, tipos compatíveis, ORDER BY só no final. Sempre!

🆚 JOIN vs UNION

JOIN: + colunas (horizontal). UNION: + linhas (vertical). Abordagens opostas

🔍 EXCEPT vs LEFT JOIN

EXCEPT: mais legível. LEFT+IS NULL: mais flexível. NOT EXISTS: mais performático

📊 Composição

UNION + EXCEPT, INTERSECT + UNION — operações podem ser encadeadas

🚀 Próxima Aula 13: Subqueries

Aula 13 — O que vem por aí:

📦 **Subqueries escalares**	Uma query dentro de outra — retorna um único valor
📋 **Subqueries de lista**	Usadas com IN, NOT IN, ANY, ALL
🔗 **Subqueries correlacionadas**	Referenciam a query externa — executam linha por linha
✅ **EXISTS e NOT EXISTS**	Verificação de existência — mais eficiente que IN
📊 **Subquery no FROM**	Derived tables — tratar resultado como tabela
💡 **Quando usar vs JOIN**	Subquery vs JOIN — trade-offs de legibilidade e performance

📝 **Para a próxima aula: conclua o Dashboard de BI e revise JOIN, GROUP BY e HAVING — são pré-requisitos para subqueries complexas.**